

C/C++ & Game Framework

겜공에서 살아남기

목차

1. C/C++
2. 게임 프레임워크

C/C++

처음 C/C++을 시작하면 아마 다음의 코드부터 접하게 됩니다.

```
#include <iostream>

int main() {
    std::cout << "Hello, World!";
}
```

이런 코드 관련한 학습은 어디에서나 찾아볼 수 있으므로 가장 기초적인 부분들은 대부분 생략하도록 하겠습니다. 그러나 중요하고 어려워 할만한 부분에서는 나름 깊게 들어가 정리해드릴 예정입니다.

시작 전 당부의 말씀으로는,

- 오류에 절대 당황하지 마십시오. - 컴파일러는 알려줄 뿐입니다.
- 궁금하면 코딩해보세요 - 예상과 다를 때 배우게 됩니다.
- AI는 알고 사용하세요 - 알기도 전에 사용할 경우, 나에게 남는게 없습니다.
- 코드의 메모리 구조는 자주 그려보세요 - 메모리 이해가 빨라질 수록 고수가 됩니다.

C/C++

[C++은 메모리를 다루는 언어이다.]

제 개인적으로 C++을 공부하면서 다음과 같은 생각을 많이 했습니다. C++은 메모리를 다루는 언어이다. 실제로 1학년들의 경우 현재 파이썬을 배우는 중인데, 파이썬에서 정수형은 몇 바이트고 ~ 문자열은 어떻게 저장되고 ~ 하는 부분들 전혀 경험해 본적이 없을 겁니다.

그러나 C++에서는 포인터 개념을 통해 메모리를 직접적으로 다룹니다. 물론 다른 언어에서 메모리를 다루지 않는다는 뜻은 아닙니다. 다만 다른 언어의 경우 언어가 메모리 관련을 알아서 처리해주는 반면, C++의 경우 우리가 직접 책임지고 다뤄줘야 합니다. 그만큼 더 어렵습니다.

어? 그럼 다른 쉬운 언어들도 있는데 왜 대부분 C++로 게임을 만드나요?

이유는 다음과 같습니다.

다른 언어들의 경우 언어 자체에서 메모리 관리를 해주다 보니 그만큼의 성능을 더 잡아먹거나 속도가 느린 경우가 많습니다. 실제로 C++의 경우 대부분의 최적화가 속도에 맞춰져 있습니다.

그렇기에 실시간 상호작용 프로그램인 게임에서는 더할 나위 없이 잘 맞는 언어입니다.

또한 게임공학과에서 메인은 C++입니다.

솔직하게 말해서 C++을 못한다? 많이 힘들겁니다. 많이 가혹할겁니다.

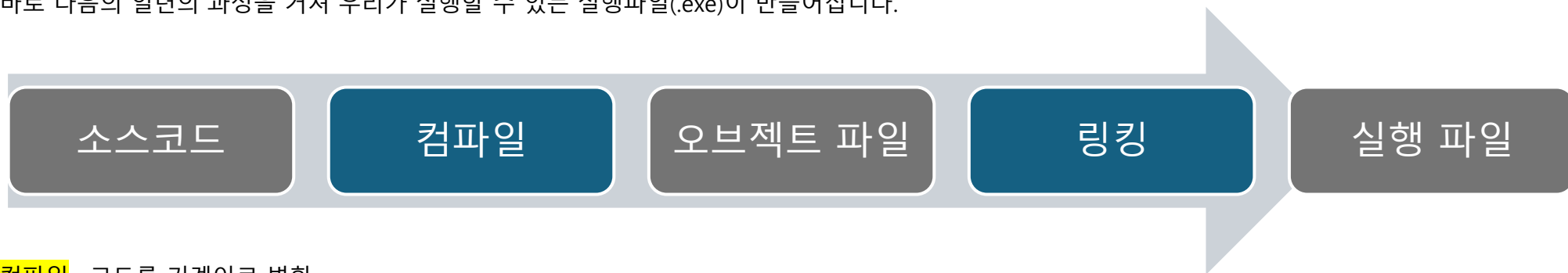
마음 굳게 먹고 따라오십쇼.

C/C++

[프로그램은 어떻게 만들어지는가]

앞으로 우리는 수많은 코드를 작성하게 될 겁니다. 그러나 이 코드들은 바로 실행되지 않습니다.

바로 다음의 일련의 과정을 거쳐 우리가 실행할 수 있는 실행파일(.exe)이 만들어집니다.



컴파일 : 코드를 기계어로 변환

링킹 : 기계어로 변환된 코드 조각과 필요한 라이브러리들을 결합

위의 과정을 "빌드" 라고 합니다.

C/C++

[컴파일 타임 vs 런타임]

앞으로 우리가 코딩을 하면서 수많은 오류들을 마주치게 됩니다. 이때 중요하게 작용하는 개념입니다.

프로그램에는 두 개의 시간이 존재합니다.

- 컴파일 타임

- 코드 변환 단계

- 문법 오류 발생

- 런타임

- 프로그램 실행 단계

- 논리/메모리 오류 발생

컴파일 타임에 오류가 생긴다면 대부분 코드 자체 문제이고,

런타임에 오류가 생긴다면 대부분 로직문제 혹은 실행 중 상태 문제입니다.

이를 통해 오류가 언제 터졌는지를 보고, 원인을 바로잡는 '디버깅'을 수월하게 할 수 있습니다.

C/C++

[프로그램 시작 = 메모리 사용 시작]

우리가 코드를 작성하고 빌드하여 실행파일을 만들었다고 생각해봅시다. 그럼 당연히 실행해보아야겠죠?
프로그램 실행의 과정은 다음과 같습니다.

빌드된 실행파일(.exe)의 경우 **저장장치**(HDD or SSD)에 저장되어있습니다.

우리가 실행하면 Window(**OS**)는 실행파일을 **메모리**(RAM)에 **공간을 할당**하여 올리게 됩니다.

이후 메모리에 모든 데이터가 올라갔다면, 우리가 작성해둔 **코드대로 실행**되게 됩니다.

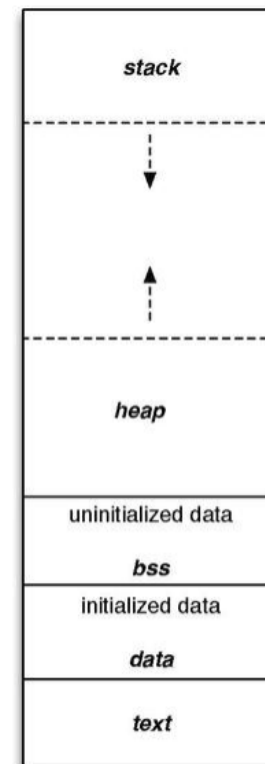
이때 할당되는 메모리는 그림과 같은 구조를 형성합니다.

STACK HEAP(Free Store) DATA CODE 총 네 부분으로 나뉩니다.

*추후 초인터 개념부터 메모리 구조가 중요해집니다. 기억해두세요

*2학기때 만날 윤정현 교수님의 수업을 위해 HEAP이 아닌 Free Store로 기억해두세요

프로그램이 모두 메모리에 올라간다? -> **프로그램은 코드/명령에 따라 메모리를 읽고 쓸 뿐입니다.**



C/C++

[변수의 본질]

이전 슬라이드에서 프로그램은 메모리를 읽고 쓸 뿐이라 했는데 어떤 메모리를 읽고 쓰는거예요?

대부분의 경우에는 변수를 읽고 쓰는 편입니다.

그럼 변수는 무엇이나?

수학에서 들어본 변수와 같습니다. 변할 수 있는 값입니다.

컴퓨터에서 의미하는 변수는 다음과 같습니다.

변수 = 메모리 공간(타입) + 이름

그림의 코드를 해석해보도록 합시다. 다음은 **변수를 '선언'** 하는 코드입니다.

```
int a{ 10 };
```

➔ OS야, int 크기 만큼의 메모리를 할당해줘. 그리고 그 메모리에는 10이라는 값을 작성해줘.

이처럼 변수는 값이 아니라 **메모리 공간의 이름**이라고 생각하는 게 좋습니다.

그리고 각 메모리 공간은 값을 담고 있습니다.

```
#include <iostream>

int main() {
    int a{ 10 };
    std::cout << "a : " << a << std::endl;
    a = 30;
    std::cout << "a : " << a << std::endl;
}
```

Microsoft
a : 10
a : 30
C:\Users\...
니다

C/C++

[메모리 생성과 수명]

자 우리는 이제 변수가 메모리 공간이라는 것을 알았습니다. 그럼 이전에 배운 내용과 조금 융합해봅시다.

우리가 만든 프로그램도 메모리에 올라간다고, 즉 할당된다고 했는데. 변수도 메모리에 할당되네요?

네. 프로그램과 변수 모두 메모리에 할당됩니다. 여기서 자세한 설명을 통해 개념적립을 하고 갑시다.

프로그램은 어느 메모리에 존재하는가?

프로그램을 보는 관점에 따라 말이 안되기도 하는 질문이지만, 지금은 우리가 작성한 코드를 프로그램이라고 생각합시다.

프로그램을 실행하면, OS가 메모리를 할당하고, 네 부분의 메모리 구조를 만듭니다.

프로그램은 이중 CODE 부분에 저장됩니다.

그럼 변수는 어느 메모리에 존재하나요?

변수의 경우 어떤 변수냐에 따라 할당되는 구역이 달라집니다. 정확히 알기 위해 다음을 알고 가도록 합시다.

[전역변수] [지역변수] [포인터]

C/C++

[메모리 생성과 수명 – 전역변수 / 지역변수 / 포인터]

해당 개념을 접근하기 전 코드의 지역에 관한 이해가 필요합니다.

기본적으로 C/C++은 다음과 같은 기본 코드를 가집니다. Main함수 범위 안에 있는 코드들이 실행되는 방식이죠.

이때 main의 뒤에서 열린 {} 해당 괄호가 지역을 나타내게 됩니다. 여기서는 main 범위/지역이 되는 것이죠

그럼 지역은 main으로만 구분되나요?

아닙니다. {}로 묶이게 되는 모든 부분이 지역이 됩니다. 이 부분은 직접 코딩해가며 체화하실 수 있습니다.

지역 예시) main으로 묶인 지역 / main 내부의 지역 / add 함수 지역

```
#include <iostream>

int main() {
    int a{ 10 };
    std::cout << "a : " << a << std::endl;
    a = 30;
    std::cout << "a : " << a << std::endl;
}
```

```
#include <iostream>

int main() {
    int a{ 10 };
    std::cout << "a : " << a << std::endl;
    {
        a = 30;
        std::cout << "a : " << a << std::endl;
    }
}
```

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    int a{ 10 }, b{ 20 };
    std::cout << "sum : " << add(a,b) << std::endl;
}
```

```
#include <iostream>

int main() {
    std::cout << "Hello, World!";
}
```

C/C++

[메모리 생성과 수명 – 전역변수 / 지역변수 / 포인터]

지역에 관한 부분이 이해가 되었다면 전역변수부터 살펴봅시다.

전역변수의 경우 지역 밖에서 선언된 변수를 의미합니다.

그림에 있는 코드에서는 global이 전역변수가 되겠습니다.

저희 그런데 메모리 구조에 대해서 이야기 하고있지 않았나요?

네. 맞습니다. 이전에 우리는 프로그램, 즉 우리가 작성한 코드가 할당받은 메모리의 CODE구역에 저장된다는 것을 살펴 봤죠.

전역변수는 DATA구역에 할당됩니다.

DATA구역에 저장되기 때문에 전체 메모리가 사라지기 전까지 전역변수를 사용할 수 있습니다.

이때 전체 메모리가 사라진다는 것은 프로그램을 종료하여 RAM에서 할당 받았던 메모리를 반환하는 것을 의미합니다.

정리하면,

전역변수는 프로그램의 시작과 같이 저장되어 프로그램 종료까지 사용이 가능합니다.

```
#include <iostream>

int global{ 10 }; //전역변수

int main() {
    int local{ 20 }; //지역변수
    int* dynamic = new int(30); //포인터
}
```

C/C++

[메모리 생성과 수명 – 전역변수 / 지역변수 / 포인터]

이번에는 지역변수에 대해 살펴보도록 합시다.

우선 우리는 {}로 묶인 부분이 지역이라는 것을 배웠습니다. 그렇다면 지역변수가 무엇인지 감이 오시나요?

지역변수는 지역 내에서 선언되어 지역의 종료와 함께 메모리에서 사라집니다.

그럼 메모리 구조의 네 구역 중 어디에 할당되나요?

메모리 구조에서는 STACK 구역에 할당됩니다.

STACK의 경우 앞으로 우리가 배울 자료구조의 스택과 같은 말입니다.

지금 상황에 맞게 간단하게만 소개하면 FILO만 기억하시면 됩니다. 컵 안에 물건을 하나씩 넣으면 가장 마지막에 넣은 물건부터 꺼낼 수 있듯이,

First In Last Out의 구조를 가지고 있습니다.

지역이 열렸다는 정보가 스택에 담기고 그 정보위에 변수가 저장되게 됩니다. 이후 지역이 끝나게 되면 스택에서 지역이 열렸다는 정보를 꺼내야하는데 변수 정보가 먼저 있게 되죠? 그래서 변수를 꺼내고 지역을 종료하게 됩니다.

수많은 지역이 열리고 닫혀도 모두 동일하게 STACK위에서 쌓이고 사라집니다.

```
#include <iostream>

int global{ 10 }; //전역변수

int main() {
    int local{ 20 }; //지역변수
    int* dynamic = new int(30); //포인터
}
```

C/C++

[메모리 생성과 수명 – 전역변수 / 지역변수 / 포인터]

이번에는 C/C++에서 가장 악명높은 초보자 절단기 **포인터**를 살펴봅시다.

우선 이전에 살펴본 전역변수와 지역변수의 경우 DATA와 STACK에 저장됩니다.
이때 **DATA**구역은 프로그램이 시작하며 정해지기때문에 **늘어나지 않는 구역**입니다.
그리고 **STACK**의 경우 **최대 크기**가 정해져 있어, 한계를 넘어가게 되면
STACK overflow라는 오류를 발생시킵니다.

어? 그럼 우리가 만들 게임은 수많은 NPC와 유저 데이터들이 필요할텐데 어디에 저장해야되요?

그래서 우리는 늘어날 수 있는 메모리 구역인 **HEAP(Free Store)**를 사용할겁니다.

이때 주의해야 할 점이 있습니다. 이전에 DATA와 STACK에 변수를 할당받을 때는 할당받은 메모리에 이름을 붙일 수 있었습니다.
모두 컴파일 타임에 할당받을 것이 결정되었기 때문이죠.

그러나 HEAP의 경우 런타임 중에 할당받습니다. 결국 HEAP에서 할당받은 메모리에 대해 미리 이름을 붙이지 못합니다.

이름을 붙이지 못하면 우리는 할당받은 메모리 공간을 어떻게 사용해야되요?

그래서 등장한 개념이 **포인터**입니다.

```
#include <iostream>

int global{ 10 }; //전역변수

int main() {
    int local{ 20 }; //지역변수
    int* dynamic = new int(30); //포인터
}
```

C/C++

[메모리 생성과 수명 – 전역변수 / 지역변수 / 포인터]

메모리의 경우 모두 1바이트 마다 고유의 주소값을 가집니다.

결국 HEAP구역도 고유의 주소값은 있는겁니다.

아 그렇다면 HEAP에서 할당받은 메모리의 주소값을 저장하여 사용하면 되겠군요?

맞습니다. 그래서 오른쪽 그림의 코드를 살펴보면 `int*`(정수형 포인터) 변수를 선언하고, 즉, 주소값을 저장할 메모리를 할당받고, 그 할당받은 공간에 `new int(30)`으로 HEAP 구역에 할당받은 메모리의 주소를 저장하는 것입니다.

정리하면,

포인터는 주소를 저장하는 변수이다.

+

포인터에 동적으로 할당받은 변수의 주소를 저장하여 HEAP구역의 메모리 공간을 사용한다.

그럼 HEAP에 할당받은 메모리의 수명은 어떻게 되요?

HEAP에 할당받은 메모리의 공간의 경우 우리가 직접 `new`로 할당받은 만큼 우리가 책임지고 해제해주어야 합니다. 결국 메모리 수명은 개발자에게 달려있습니다.

우리가 직접 관리해주어야 한다는 점이 C/C++이 어려운 이유입니다.

```
#include <iostream>

int global{ 10 }; //전역변수

int main() {
    int local{ 20 }; //지역변수
    int* dynamic = new int(30); //포인터
}
```

C/C++

[데이터는 모두 0과 1이다]

이번에는 변수, 즉 우리가 할당받아 사용하는 메모리 공간에 무엇이 저장되는지 살펴봅시다.

메모리는 전자 회로의 조합으로 내부적으로는 모두 0과 1의 '비트'만 저장합니다.
이 말은 곧 모든 데이터 정보는 이진수로 저장되어있다는 말과 같습니다.

그럼 각 이진수가 문자를 나타내는지 숫자를 나타내는지 어떻게 알아요?

메모리 단계에서는 모르는게 맞습니다. 그러나 우리가 코딩하는 프로그램 단계에서는 자료형을 통해 해당 이진수가 어떤 의미를 갖는지 해석합니다.

그렇다면 같은 데이터를 다른 자료형으로 해석해서 이용이 가능한가요?

네. 가능합니다. 그 예시로 다음의 그림을 참고하시면 될 거 같습니다.

01000001이라는 이진수를 char형으로는 'A', int형으로는 65라는 값으로 해석할 수 있습니다.

```
01000001
→ char : 'A'
→ int  : 65
```

앞서 말했듯이 자료형은 메모리의 크기와 데이터의 해석 방식을 결정하게 됩니다.

조금 더 나아가 연산 방식도 결정합니다.

C/C++

[자료형과 메모리 크기]

이번에는 자료형에 대해 조금만 더 파악해봅시다.

우리는 지금까지 자료형이 데이터의 해석방식을 결정한다고 배웠고, `int a{ 10 };`이 어떤 의미인지 번역하는 방법도 배웠습니다. 또 메모리 구역별로 어떤 특징이 있는지, 그중 크기에 제한이 있다는 구역도 알게 되었죠.

그런데 한가지 빠진 거 같지 않나요?

우리는 여태껏 메모리 공간 할당~, 메모리, 이진수 등 여러 가질 말했지만 이 공간의 크기에 대해서는 이야기 한적이 없습니다. 이 부분을 다뤄봅시다 .

우선 자료형이 메모리 공간에 담긴 데이터의 해석방식을 알려주기도 하지만 또 알려주는 게 하나 더 있습니다..

바로 **메모리 공간의 크기**입니다. 컴파일 하는 과정에서 코드는 OS에게 이렇게 요청하게 됩니다.

우리 이 자료형으로 쓸 변수명의 메모리 공간이 필요해.

그럼 OS는 자료형에 맞는 크기의 메모리 공간을 할당해주죠.자료형마다 할당받는 메모리의 크기는 시스템/컴파일러마다 다르지만 우리가 작업할 환경에서는 다음과 같습니다. 기본 자료형만 알려드리겠습니다. (작업환경 기준 : x64, Windows, Visual Studio, C++)

char : 1 바이트 / int : 4 바이트 / float : 4 바이트 / double : 8 바이트

C/C++

[자료형과 메모리 크기]

기본적으로 메모리 공간이 클수록 표현되는 정보는 더 정확하고 자세해지고 범위도 커집니다.
그 예시로 문자형(char)은 영어의 경우 한 글자에 1 바이트면 충분하지만, 한글의 경우 수많은 자음모음 조합으로 인해 2바이트를 소모하게 됩니다.

문자는 그래도 개수가 정해져 있는것 아닌가요?

맞습니다. 그래서 값이 무한할 수 있는 숫자의 경우 바이트 제한으로 인해 표현 가능한 범위가 존재합니다.

예시로

short의 경우 2바이트(16비트)로, $2^{16} = 65,536$ 개의 숫자를 표현할 수 있습니다. -> -32,768 ~ 32,767

int의 경우 4바이트(32비트)로, $2^{32} =$ 약 43억개의 숫자를 표현합니다. -> -2,147,483,648 ~ 2,147,483,647

위의 두 자료형의 경우 정수를 표현합니다.

이번엔 실수를 살펴볼건데 실수의 경우 소수점을 표현하기 위해 이진수를 해석하는 방법이 달라집니다.

float의 경우 단정도 부동소수점을 사용하여 표현합니다. 4바이트이기 때문에 표현 범위는 약 $\pm 1.2 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$ 입니다.

Double의 경우 배정도 부동소수점을 사용하며, 8바이트를 사용하기 때문에 더 넓은 약 $\pm 2.2 \times 10^{-308} \sim \pm 1.8 \times 10^{308}$ 를 표현합니다.

실수를 표현하는 자료형의 경우, 무한소수 혹은 이진수로 표현할 수 없는 값에 대해 근사값을 사용하기 때문에 연산과정에 오차 누적이 발생합니다.
부동소수점에 관해서는 설명해주는 자료가 많으니 꼭 한번쯤 찾아보고 공부해 보길 권장 드립니다.

C/C++

[우리가 사용할 C++]

이렇게 최대한 간단하게 필요한 베이스를 학습해보았습니다.

물론 빠지거나 모자란 내용도 있지만, 아마 자료구조나 컴퓨터 구조, 운영체제 등 다른 과목에서 융합되어 접하게 됩니다.

이제 바로 C++을 사용하며 프로젝트를 진행해보려 합니다. 마지막으로 C++은 어떻게 학습하는 것이 좋을지 제 경험에 기반하여 정리해드리겠습니다.

우선 C++의 경우 지속적으로 업데이트 되는 언어입니다. C++11부터 현재 C++23, 곧 나올 C++26까지. 각 버전들이 올라가며 과거의 문법들이 정리되고, 더 효율적인 문법으로 등장하기도 합니다. 게다가 역사도 긴만큼 C++에 관한 내용은 정말 방대합니다.

그래서 다음의 링크를 자주 참고하시기 바랍니다.

<https://en.cppreference.com/w/>

처음엔 해당 레퍼런스에서 학습하기 어려울겁니다. 그러나 윤정현 교수님의 C++강의때에도 위의 레퍼런스를 기반으로 설명하시기에 미리 연습해두는 것이 좋습니다.

또한 내용이 방대한 만큼 제가 모든 부분을 알려드리긴 어렵습니다. 다만 게임을 개발하는 과정에 있어서 필요한 C++은 적재적소에 사용할 수 있도록 알려드릴 예정입니다.

그럼 우리가 이해한 C++을 가지고 게임의 세계로 넘어가보겠습니다.

Game Framework

[프레임 워크란 무엇인가]

대부분 프레임워크라는 말은 들어봤으나, 정확히 이게 무엇인지 모르는 경우가 많았을 것입니다.

GPT를 사용한 답은 그림과 같습니다.

우리 또한 GPT의 설명처럼 앞으로 수많은 게임프로젝트를 진행하게 될겁니다. 이때 매번 새로운 구조를 가져가기 보다는 일관된 가장 대표적인 프레임워크로 효율적으로 작업하자는 의도입니다.

이때 프레임워크는 **단순 뼈대 혹은 흐름**이기에, 자신의 능력이 된다면 새로운 프레임워크를 구성하여 성능 상의 이점을 챙겨도 좋습니다. 해당 스터디에서는 가장 단순한 구조에 여러 **프로그래밍 패턴**을 결합하여 알려드리도록하겠습니다.

프레임워크란 무엇인가

프레임워크(Framework)는
소프트웨어를 만들 때 기본 구조와 규칙을 미리 만들어 놓은 틀이다.

쉽게 말하면,
개발자가 처음부터 모든 것을 직접 구현하지 않아도 되도록
전체적인 뼈대와 흐름을 제공하는 시스템이다.

왜 사용하는가

프레임워크는 다음 문제를 해결하기 위해 존재한다:

- 매번 같은 구조를 반복해서 만드는 비효율
- 코드 스타일과 구조의 일관성 부족
- 개발 속도 저하

 즉,

“이미 검증된 방식으로 빠르게 개발하게 해주는 도구”

Game Framework

[게임 프레임워크]

가장 기본이 되는 프레임워크는 다음과 같습니다.

Initialization : 게임에 필요한 모든 값과 메모리 등을 초기화 및 세팅

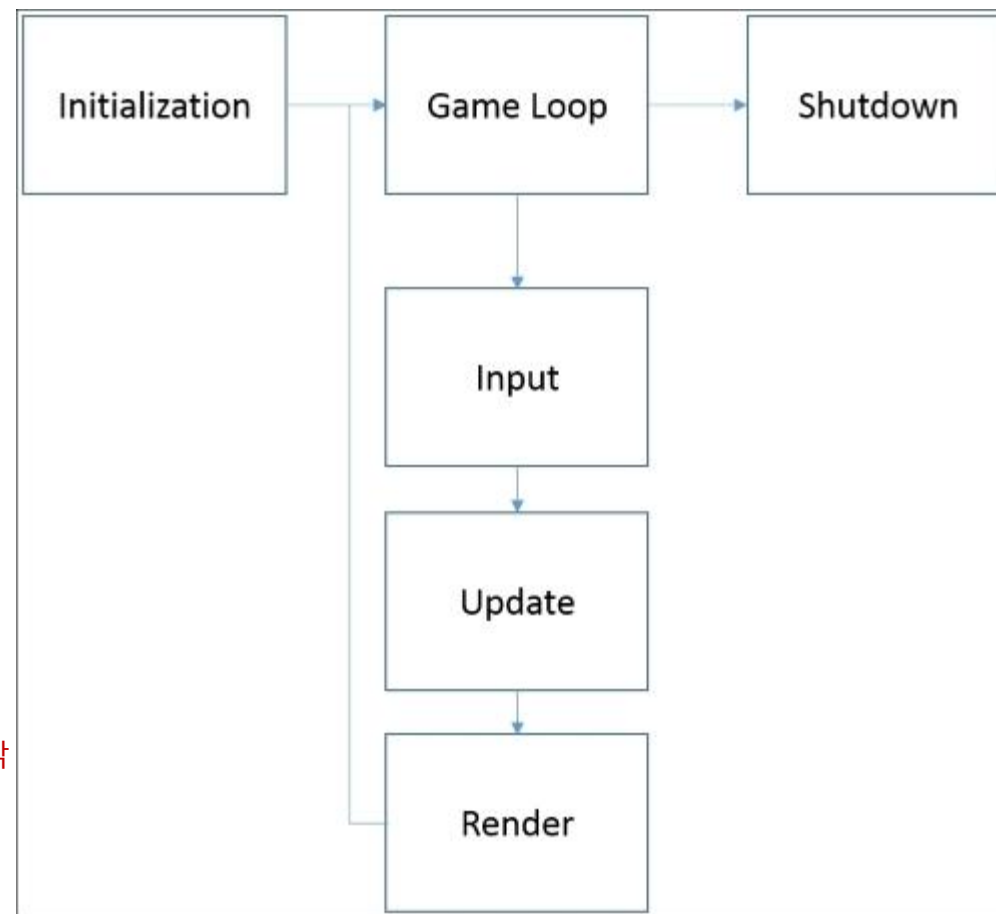
Game loop

- **Input** : 모든 입력을 받고 처리 혹은 update에 전달
- **Update** : 입력 혹은 시간흐름에 따른 변화 모두 처리
- **Render** : 화면에 그리기

Shutdown : 종료 조건에 부합할 경우 Game loop를 탈출하여 게임 종료

게임이 동작하는 흐름이나 구조는 알겠는데 이걸 어떻게 구현해요??

이제 코드를 통해 학습해보도록 합시다. 프레임 워크 구성에 따라가다보니 C++개념은 여기저기 흩어지게 될겁니다. 정말 C++을 잘하고 싶다면 흩어진 C++개념들을 조각조각 붙여 적립하는 과정은 필수입니다.



Game Framework

[게임 프레임워크 - Initialization]

해당 단어를 직역하면 '초기화'라는 뜻입니다.

앞에서는 언급한 적 없지만 코딩에서는 다음도 초기화라고 합니다

```
int a { 10 };
```

'a라고 불릴 int형의 메모리 공간을 10으로 초기화 해줘' 의 의미입니다.
이처럼 어떠한 변수를 선언할때 초기값을 주는 것을 초기화라고 합니다.

그럼 게임 프레임워크의 초기화도 같은 건가요?

네. 같습니다.

게임이 처음 시작할때 필요한 몹의 마릿수, 적의 체력, 플레이어의 공격력 등 변수나 객체를 초기화한다는 의미입니다.
또한 나중에 클라이언트 분야에서 배울 DirectX의 렌더링을 위해 필요한 객체나 여러 외부 API의 객체들도 이때 초기화합니다.

객체는 뭔가요?

이를 설명하기 위해서 잠시 한 페이지를 할당하도록 하겠습니다.

객체와 클래스

[C와 C++의 차이]

C와 C++의 가장 큰 차이점은 class입니다. (클래스에 관한 다른 내용들은 필요할 때 조금씩 설명하도록 하겠습니다)

객체를 물어봤는데 왜 클래스부터 설명해요?

객체의 시작이 클래스이기 때문입니다.

조금 더 자세하게 하자면, 클래스 라는 사용자 정의 자료형을 통해 객체들을 찍어낼 수 있습니다.

코드로 살펴보겠습니다.

(실제로 표준 자료형의 경우 클래스와 객체라고 하지 않습니다. 이해를 위한 비유입니다.)

```
int a { 10 };
```

→ int는 클래스라고 말할 수 있고, a는 해당 클래스로 만들어진 객체라고 할 수 있습니다.

실제 클래스와 객체로 표현해보겠습니다.

```
class Student{  
private:  
    std::string name{};  
    int old{};  
}
```

```
Student a{ "이태형", 26 };
```

-> 분석할 수 있겠나요? 무엇이 클래스고, 무엇이 객체인가요?

Game Framework

[게임 프레임워크 - Initialization]

우린 이제 클래스가 있고, 클래스를 사용하여 객체를 만들 수 있다는 걸 알았습니다. 이제 바로 클래스 개념을 융합하여 설명하도록 하겠습니다.

앞으로 우리가 할 프로그래밍은 대부분 기능 단위로 묶어 클래스를 구성하게 됩니다.

그럼 어떤 기능 단위로 클래스를 구성해야하나요?

우선 크게 3계층 정도로 나뉘게 됩니다. Core / System / Object

Core : 앞서 말한 흐름대로 흘러갈 수 있도록합니다. 초기화와 루프, 종료조건을 검사합니다.

System : 각종 기능을 담당합니다. Render, sound, resource 등

Object : 게임 내부의 오브젝트들(몬스터, 플레이어 등)을 찍어내는 클래스입니다.

설명만 봐서는 잘 모를 수 있습니다. 제가 예전에 진행했던 프로젝트 코드를 공유해드릴테니 해당 코드를 보고 분석하는 연습과 각 클래스가 어떤 역할을 하는지 이해하시면 될거 같습니다. 콘솔창 텍스트 형식의 게임이라 1학년 분들도 분석하기에 문제 없습니다.

<https://github.com/THe-L00t/No-One-Spoke-Today.git>

기능별로 나누는 건 알겠어요. 근데 클래스를 구성하는 방법을 아직 모르겠어요!

다시 프레임 워크 설명을 멈추고 클래스에 대해서 설명드리도록 하겠습니다.

클래스

[struct와 class]

우선 기본적인 사용자 정의 자료형부터 시작해봅시다.

사용자 정의 자료형을 알기 전까진 우린 기본 자료형인 int, char 등 숫자와 문자, 더 나아가봐야 문자열밖에 다루지 못했습니다. 하지만 프로그래밍이 발전하면서 점점 사람 데이터들, 혹은 게임의 몹 데이터 등 더 특이한 자료형을 다룰 필요성을 느끼게 됩니다.

실제로도 코딩하다보면 기본 자료형만으로 프로그램을 만들기 버거울겁니다. 그래서 등장한 개념이 사용자 정의 자료형입니다.

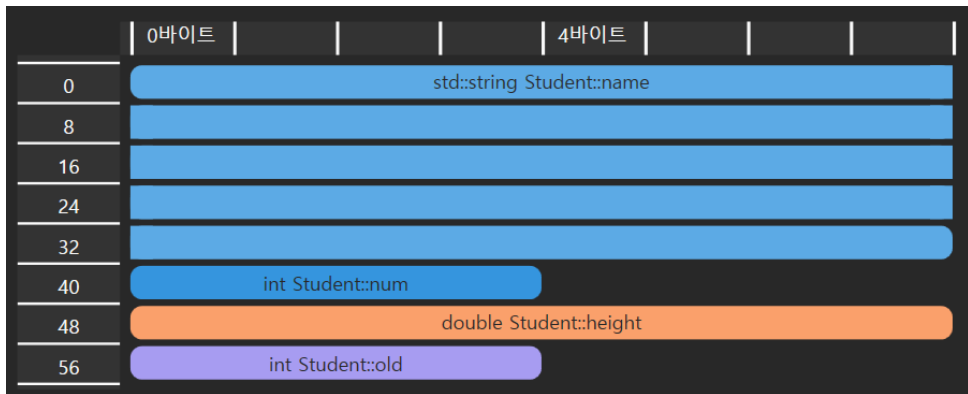
예를 들어, 우리는 몬스터를 코드 안에서 다루고 싶어합니다. 몬스터는 종류, hp, 공격력 등등 여러 값을 가지고 있습니다. 이때 각각을 일반 자료형으로 만들게 된다면, 변수의 이름만 보고 누구의 hp의 값인지 모를 수 있습니다. 이를 방지하기 위해 새로운 사용자 정의 자료형을 만들어 해당 자료형이 필요한 값들을 묶어서 관리할 수 있게 해줍니다.

그림의 예시처럼

문자열과 정수, 실수를 student라는 사용자 정의 자료형으로 포장한 것입니다. 그리고 우측의 그림은 사용자 정의 자료형을 메모리로 그린 그림입니다.

```
class Student {  
private:  
    std::string name;  
    int num;  
    double height;  
    int old;  
};
```

이때 중요하게 보아야할 것이 우측 그림에 있습니다. 메모리들 사이 빈칸이 보이시나요? 이를 메모리 패딩 padding 이라고 합니다. 나름 중요한 내용이니 개인적으로 더 알아보길 권장합니다.



클래스

[struct와 class]

사용자 정의 자료형에는 두 가지가 있습니다.

Struct 와 **Class**, 이 둘은 1.5개의 부분을 제외하고는 모두 동일합니다.

설명하기 쉬운 0.5부터 설명하도록 하겠습니다. (그림을 같이 참고하면 좋습니다.)

```
class Student {  
    |   std::string name;  
private:  
    |   double height;  
public:  
    |   int old;  
};
```

```
struct Professor {  
    |   std::string name;  
public:  
    |   int old;  
private:  
    |   std::string major;  
};
```

우선 struct의 경우 기본 접근 지정자가 public입니다. 접근 지정자를 따로 설정하지 않는 경우 struct의 모든 멤버 변수에 접근이 가능합니다. Class의 경우에는 기본 접근 지정자가 private입니다. Public으로 설정해주지 않으면 멤버 변수에 접근이 불가능 합니다.

어 멤버변수는 뭐고, 접근한다는건 어떤 의미인가요?

우선 멤버의 개념은 위의 두 그림처럼 어떠한 사용자 지정 자료형이 **소유하게 되는 변수 또는 함수**를 멤버라고 합니다.

접근한다는 것은 해당 클래스 혹은 구조체의 멤버를 클래스 혹은 구조체의 **외부에서 호출/엑세스** 하는것을 말합니다.

```
void main() {  
    Student s{};  
    Professor p{};  
  
    std::cout << s.name << p.name << std::endl;  
    std::cout << s.old << p.old << std::endl;  
}
```

앞서 설명했듯이 class인 student의 name에는 다음과 같은 빨간줄이 표시됩니다.

abc E0265

멤버 "Student::name" (선언됨 줄 5)에 액세스할 수 없습니다.

learning

main.cpp

24

클래스

[struct와 class]

0.5의 차이는 간단한데, 그럼 1의 차이점은 어떤가요?

1의 차이부터는 이제 매우 큰 차이를 보입니다.

아마 인터넷에 클래스를 검색해보면 OOP나 객체지향, 상속, 캡슐화 등등 여러 키워드가 나올겁니다. 이 모든 키워드가 중요합니다.

내용이 너무 많고 중요하기 때문에 개인적으로 검색해보거나, 프로젝트를 진행하며 필요한 부분을 설명할 때 개념을 잘 적립하시기 바랍니다.

아마 프로젝트를 진행하고, 스터디를 진행하면서 다음의 키워드 순서로 진행하게 될 거 같습니다.

클래스의 기본 – rule of five

클래스의 상속

함수 오버로딩과 연산자 오버로딩

그럼 다시 프레임 워크 설명으로 돌아가보겠습니다.

Game Framework

[게임 프레임워크 - Initialization]

Core 객체부터 간단하게 설명하도록 하겠습니다.

이전에 설명했듯이 해당 객체는 프레임워크를 전반적으로 담당합니다.
이때 객체의 이름은 개발자 자유입니다 ^^

아무튼 전반적으로 담당하기에 해당 객체는 다음의 3개 함수를 가지는 편입니다.

init(), loop(), shutdown() 각각의 함수 내에서 여러 게임에 필요한 시스템 객체 초기화와 루프 반복, 종료 조건을 만족하면 여러 객체 정리 후 종료까지 진행합니다.

```
class Core {  
public:  
    void init();  
    void loop();  
    void shutdown();  
private:  
    // 여러 시스템 객체들을 소유 혹은 관리  
};
```

여러 시스템 객체들을 소유 혹은 관리 라는 주석은 어떤 의미예요?

게임에서는 사운드, 게임 로직, 렌더링, 입력처리, 리소스 로드 등 각 기능을 담당할 객체가 필요합니다. 이를 core객체가 관리할 수 있도록 멤버 변수로 소유하거나, 포인터를 통해 각 시스템 객체들에 접근 할 수 있도록 합니다.

*여기까지의 내용을 직접 해보면서, 파일분할과 클래스 사용 등 추가 설명을 진행하도록 하겠습니다

Game Framework

[게임 프레임워크 – Game Loop]

자 앞선 학습과 실습을 통해 필요한 시스템 객체들을 모두 초기화 해주었습니다.
이제는 게임이 돌아갈 수 있도록 해야합니다.

게임루프 안에서 우리의 입력을 처리하고, 오브젝트들의 위치나 충돌 이벤트를 처리하고, 이후에 그리기까지.
우리가 초기화 했던 시스템을 이용하여 로직을 구현하면 됩니다.